



FINANCE TRACKER

FINAL PROJECT

TEAM



**Seitmukhambetov
Alikhan**



Bolat Mubarak



**Yesenaliev
Arystanali**



Tselichshev Maxim

THE PROBLEM

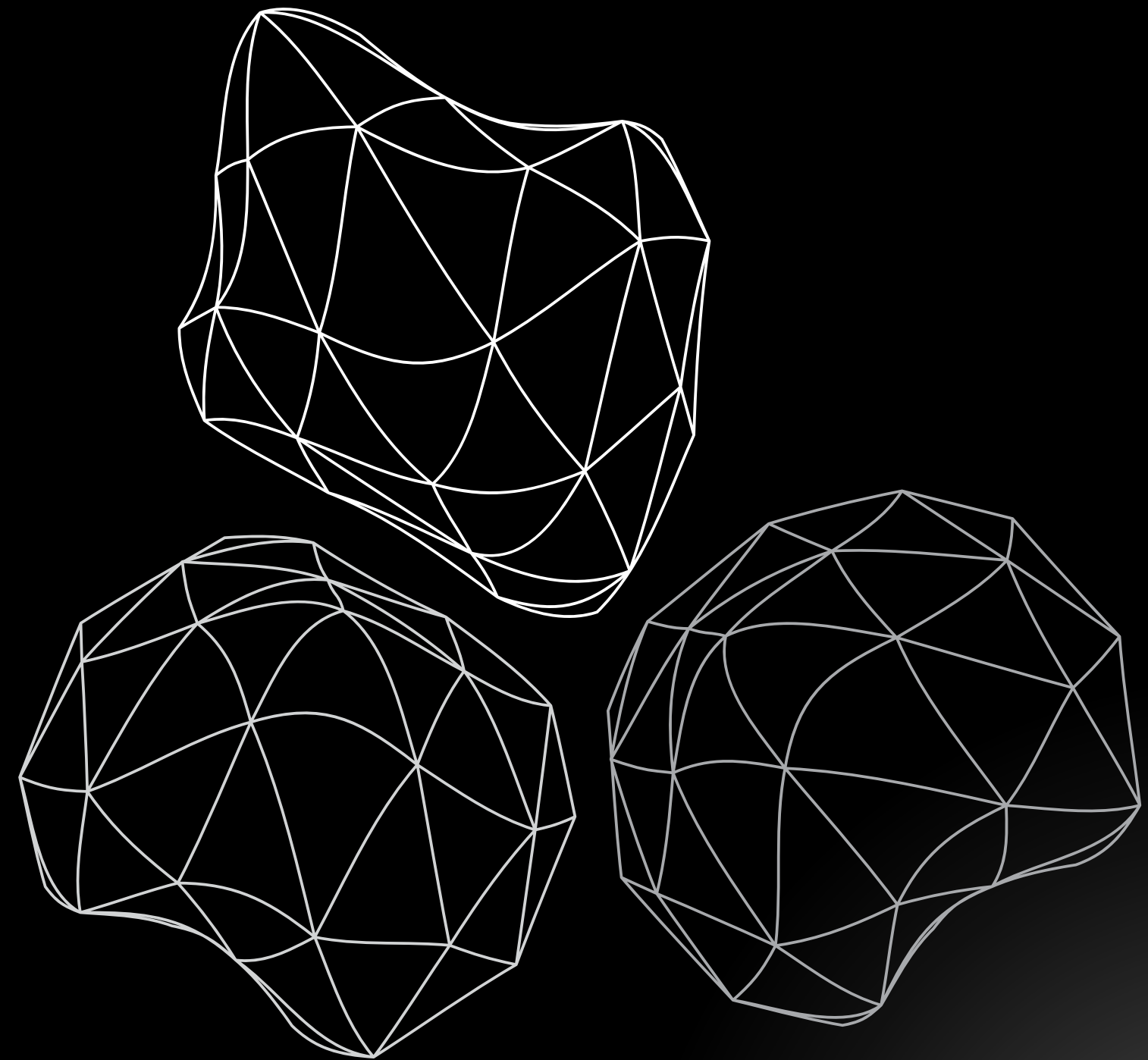
Many people struggle to track their personal finances and have no clear understanding of where their money goes

TRADITIONAL APPROACH

Manual tracking is tedious, error-prone, and lacks real-time insights

OUR SOLUTION

An automated finance tracker that seamlessly monitors income and expenses



itp2-final-1/

data/

transactions.json

models/

account.py

categories.py

interface.py

transactions.py

services/

analytics.py

storage.py

tests/

test_analytics.py

test_validators.py

utils/

decorators.py

validators.py

main.py

SYSTEM ARCHITECTURE

**models/ – Data structures
(Transactions, Accounts, Categories)**

**services/ – Business logic and financial
calculations**

tests/ – Automated test suites

main.py – Application entry point

APPLICATION DEMO

=== Personal Finance CLI ===

- 1) Add transaction
- 2) Show balance
- 3) Show JSON records (storage)
- 4) Monthly report (analytics)
- 5) Category breakdown (analytics)
- 6) Detect overspending (analytics)
- 7) Large expenses (analytics)
- 8) Set category limit
- 0) Exit

Choose action: —————>

5 [LOG] Start: show_category_breakdown
food: 23650
transport: 1040
entertainment: 15500
health: 3400
other: 1200
[LOG] Done: show_category_breakdown
(0.0002s)

2 [LOG] Start: show_balance
Current balance: 134206.0
[LOG] Done: show_balance (0.0001s)

3 [LOG] Start: show_json_records
Records in JSON: 15
{'amount': 41898, 'category': 'stipend', 'type': 'income', 'date': '2026-01-12'}
{'amount': 4500, 'category': 'food', 'type': 'expense', 'date': '2026-01-15'}
.....
[LOG] Done: show_json_records (0.0026s)

6 [LOG] Start: show_overspending
Overspent categories: food, transport,
entertainment, health, other
[LOG] Done: show_overspending (0.0002s)

7 [LOG] Start: show_large_expenses
Minimum amount for large expense: 4500
entertainment: 12000
food: 15000
[LOG] Done: show_large_expenses
(4.6025s)

1 [LOG] Start: add_transaction
Amount: 200
Category: Salary
Type (income/expense): income
Date YYYY-MM-DD (Enter = skip):
Transaction added. Balance: 134206.0
[LOG] Done: add_transaction (13.1906s)

4 [LOG] Start: show_monthly_report
2026-05: income=178796, expenses=44790
[LOG] Done: show_monthly_report (0.0004s)

8 [LOG] Start: set_category_limit
Category name: food
Spending limit: 200
Limit for 'food' set to 200.0
[LOG] Done: set_category_limit (9.0140s)

0 Bye!



DATA STRUCTURES & RATIONALE

Dictionary (dict) in category_expenses()

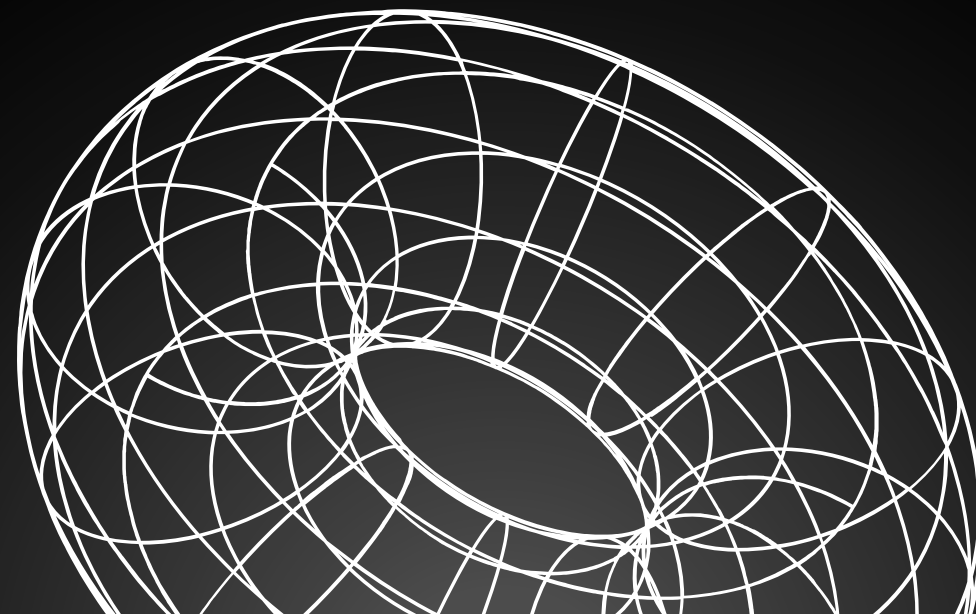
Why: Used for grouping expenses by category. Provides $O(1)$ fast lookup instead of inefficient $O(n)$ linear searches

List for Transactions

Why: Chronological order matters. Supports quick appending and seamless iteration over financial history

Set for Unique Categories

Why: Guarantees unique values with zero duplicates for distinct filtering



ADVANCED PYTHON FEATURES

Generators in `monthly_summary()`

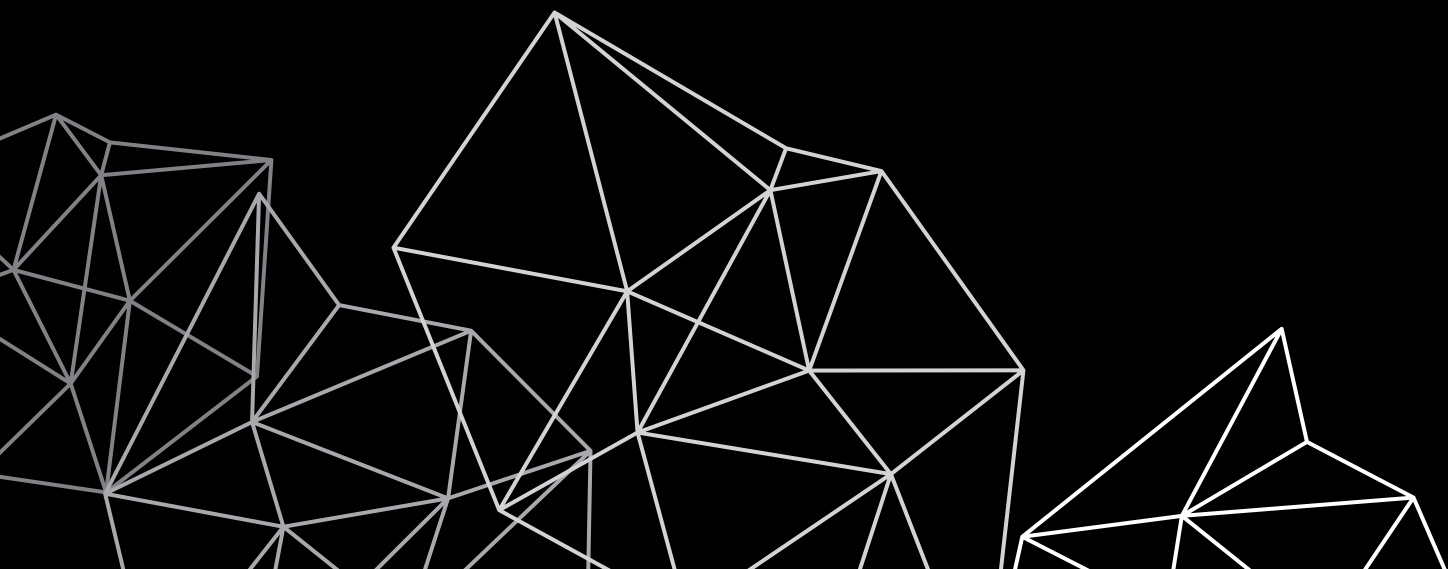
Why: Memory-efficient data streaming instead of loading huge financial datasets into memory all at once

Filter & Lambda in `large_expenses()`

Why: Provides cleaner, more declarative, and optimized code compared to traditional for loops

Custom `@log_action` Decorator

Why: Automates transaction logging, tracks execution history, and enhances debugging



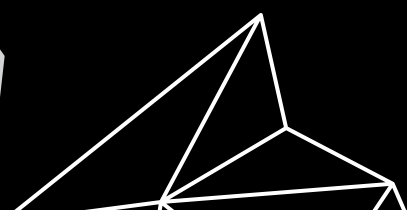
OBJECT-ORIENTED PROGRAMMING

Inheritance: Income and Expense inherit core properties from the base Transaction class

Encapsulation: Private attributes, protect crucial data inside Account, Category, and FinanceAnalyzer

Polymorphism: The apply() method is overridden to exhibit distinct behaviors for Income and Expense

Abstraction: Transaction is implemented as an Abstract Base Class to enforce a strict contract



TESTING STRATEGY

**Test Execution: 100% successful test coverage via
pytest (All tests passed)**

What We Tested:

**Standard Cases: Routine transaction logging and baseline
calculations**

**Edge Cases: Empty financial history, exact budget limits,
and extreme overspending scenarios**

TEAM CONTRIBUTION



Seitmukhambetov
Alikhan

ANALYTIC BLOCK (CLASS FINANCE ANALYZER WITH MONTHLY_SUMMARY, CATEGORY_EXPENSES, DETECT_OVERSPENDING, AND LARGE_EXPENSES METHODS), TESTS FOR FINANCE ANALYZER



Bolat Mubarak

MODELS (TRANSACTION.PY, CATEGORIES.PY, ACCOUNTS.PY AND INTERFACE ACCOUNTABLE), TESTS



Yesenaliev
Arystanali

STORAGE BLOCK (STORAGE.PY, TRANSACTIONS.JSON)



Tselichshev Maxim

UTILS MODULE (@LOG_EXECUTION DECORATOR, VALIDATORS.PY), MAIN, PRESENTATION



THANK YOU

SE-2525